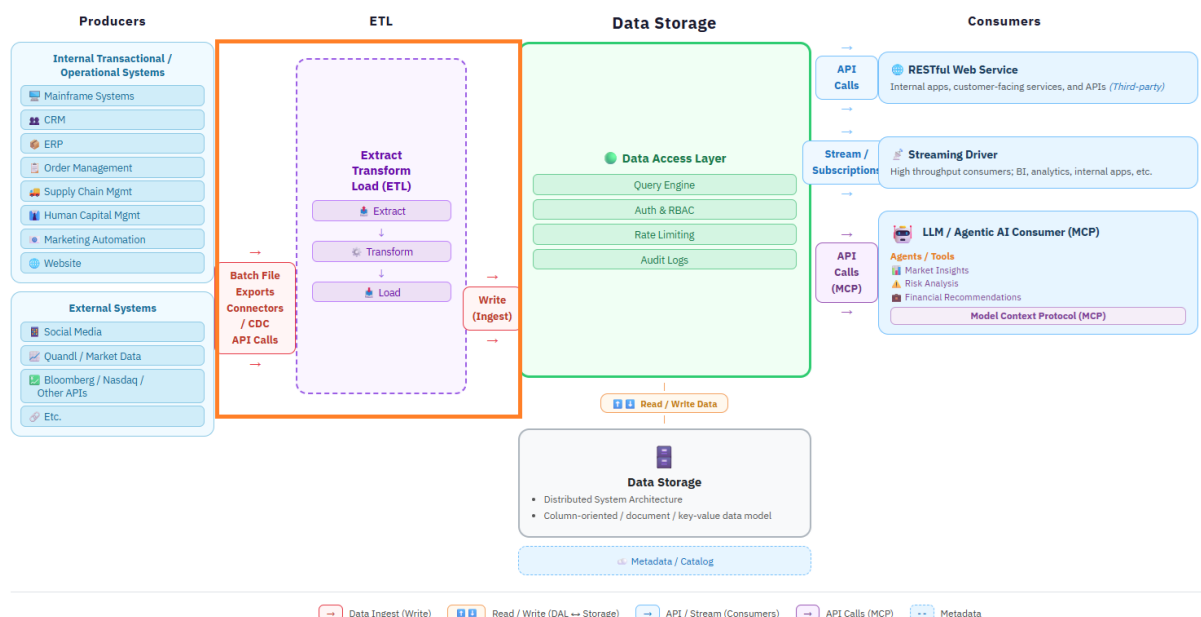


The Data Ingestion Process

The Data Ingestion Process	1
Data Ingestion Overview	1
Stages of the Ingestion Process	2
Designing the Ingestion Flow	2
Requirements for Ingestion Process	2
Implementation Hints	3
Expected Deliverables	3
References	4

Data Ingestion Overview

In plain terms, data ingestion is the part of the platform that goes outside the warehouse, collects and stores data in a form the rest of the system can use. This means taking financial market data from external providers, aligning it with the internal model, and making sure it stays trustworthy over time.



Once you already have a clean data access layer created in previous week, ingestion becomes the controlled entry point through which new data enters the platform. The point is not to tie you to a particular tool, but to help you design a flow that would still make sense if the provider, storage engine, or programming language changed.

Learning outcomes

- Understand extraction, transformation, and loading as separate responsibilities.
- Design a provider-agnostic ingestion flow for time-series financial data.
- Preserve provenance and handle changing attributes without breaking the model.
- Make ingestion repeatable, safe, and easier to debug.

For this lab, a batch-style source that exposes historical or delayed market data is more than enough. You do not need a real-time stream to demonstrate the ingestion principles well.

Stages of the Ingestion Process

A well-designed ingestion flow usually has three stages:

- **Extraction** - read data from the external source. Depending on your design, raw responses may be kept temporarily in a staging area (implemented in the database or in-memory depending on the requirements/design), or processed directly if that is enough for your workflow.
- **Transformation** - convert the provider-specific structure into the internal warehouse representation. This is where you normalize identifiers, reshape attributes, convert data types, and validate records.
- **Loading** - store the transformed data and the related metadata in the warehouse. Loading can happen after transformation completes, or progressively while records are being transformed.

The important idea is separation of concerns (Single Responsibility Principle). Each stage has a different job, even if your final implementation decides to combine some of them for simplicity.

Designing the Ingestion Flow

The ingestion flow will usually start from a small list of financial instrument identifiers. For each identifier, the system requests data from an external provider, processes the returned records, and stores them as assets, data-source metadata, and time-series data.

External providers often return large datasets in chunks or pages. Your design should therefore be able to continue fetching until there is no more data, instead of assuming everything arrives in one response.

Suggested ingestion workflow

- Input -> list of asset identifiers

 1. Fetch data from the provider
 2. Continue through all returned pages/chunks (be MINDFULL at paged endpoints)
 3. Identify the source and the available attributes
 4. Map source records to the internal canonical model
 5. Create missing metadata safely
 6. Store time-series records and capture provenance
 7. (Optional) Report what was loaded, skipped, or retried

Requirements for Ingestion Process

The real design questions appear when the same data is ingested twice, when metadata already exists, or when multiple ingestion jobs run at the same time.

- Treat **create-if-missing** logic carefully. Avoid fragile check-first-then-insert flows if concurrent jobs could interleave them.

- Prefer idempotent behaviour. Re-running the same ingestion should not corrupt data or create inconsistent duplicates.
- Record the source provider and enough context to trace where each dataset came from.
- Different sources - and even the same source over time - may expose different attribute sets.
- Handle partial failures explicitly so the job can be retried or resumed with confidence.

In short, your ingestion pipeline should be correct not only when it works once, but also when it works many times.

Implementation Hints

- Keep extraction, transformation, and loading in separate modules or components. Give each part one clear responsibility.
- Use the data access layer as the persistence boundary, so the ingestion process does not leak database-specific logic into the rest of the platform.
- Do not hard code every incoming field if the source is likely to evolve! Derive attribute names from the incoming dataset and store them as metadata at data source entity.
- Validate and normalize incoming data early: dates, numeric values, identifiers, and missing fields should be handled on purpose.
- Design a scalable ingestion pipeline, considering the following:
 - Batch operations for database writes – reduce database roundtrips
 - Asynchronous job processing – asynchronous processing, message queues (RabbitMQ) or streaming platforms (Apache Kafka) decouple producers from consumers, improve fault tolerance and enable workloads distribution
- Add observability from the beginning: count fetched records, transformed records, stored records, skipped records, and failures. These statistics should be made visible to the end-use.
- Nasdaq Data Link [NDL, NDL API] can be used as financial data provider in this project. It provides both free and premium data – collected from a vast variety of organisations – which may be accessed through one of the following Nasdaq Data Link APIs:
 - Streaming API for real-time data [STREAMING]
 - REST API for real-time or delayed data [REALTIME]
 - REST API for tables data [TABLE DATA] (recommended approach)
- Nasdaq Data Link endpoints use pagination to allow access to large amounts of data (see `qopts.cursor_id` parameter).
- **Bitfinex Crypto Coins Exchange Rate** [BITFINEX] is a free cryptocurrency exchange rates featuring high, low, mid, last, bid, ask, and volume metrics as presented by the Bitfinex website. Nasdaq [NASDAQ]

Expected Deliverables

- An ingestion module that can import external financial time-series data for a small set of instruments.

- Successful runs of the import flow that will persist assets, data-sources, and time-series records in the data warehouse; these are paramount for next features we shall develop in upcoming weeks.
- Basic unit tests.

References

[NDL] <https://data.nasdaq.com>,

[API] <https://www.nasdaq.com/solutions/data/nasdaq-data-link/api>

[STREAMING] <https://docs.data.nasdaq.com/docs/streaming-api-1>

[REALTIME] <https://docs.data.nasdaq.com/docs/api-for-real-time-or-delayed-data-1>

[TABLE DATA] <https://docs.data.nasdaq.com/docs/api-and-analysis-tools-for-tables-data>

[BITFINEX] <https://data.nasdaq.com/databases/BITFINEX>